SPIRIT GATE

Protótipo visual 2D — relatório técnico de alterações e validação

Versão: 0.1 (arena de debug local) **Data:** 17/10/2026

Stack mantida: Java 17, Maven, Kryonet e PostgreSQL (rede/banco existentes continuam separados).

Objetivo: permitir testar imediatamente movimento, colisões e combate com feedback visual, reutilizando as regras reais do domínio.

1. Resultado entregue

- **Janela Swing/Java2D** com arena 960 × 640, HUD, 3 inimigos, barras de HP, números de dano e telemetria.
- **WASD** com 8 direções e velocidade diagonal normalizada; cenário com limites e três obstáculos sólidos.
- **Hitboxes reais** de círculo/círculo e círculo/retângulo; visualização opcional com a tecla H.
- **Combate básico** com alcance, cone frontal de 90°, cooldown de 0,38 s e aplicação de dano via CombatService.
- **IA mínima** com perseguição e contra-ataque; morte, vitória e reinício da arena pela tecla R.

Importante: esta entrega é uma simulação local autoritativa de gameplay, não um MMORPG online pronto. Não integra a sessão de login nem realiza multiplayer.

2. Prévia real do programa

Captura obtida ao executar a janela gráfica em um servidor de tela de teste, após entradas simuladas de teclado. O HP 94/100 na prévia é consequência do ataque de um mob durante a simulação.



Figura 1 — Arena 2D em Java2D, com hitboxes habilitadas, barra de vida e três adversários.

3. Arquitetura e fluxo de dados

A interface não edita diretamente as entidades. O estado do jogo tem um único proprietário: a thread de simulação.

- **DebugLauncher (Swing EDT)** captura WASD/Espaço/H/R.
- **DebugInput** transmite intenção (direções + ataque), nunca a posição final do personagem.
- **DebugRuntime** consome a intenção em uma thread exclusiva de 30 Hz.
- **DebugWorld** verifica colisões, alcance e cooldown antes de chamar CombatService.
- **PlayerFactory / MobFactory / VitalityService** criam os atores e definem o HP.
- **DebugSnapshot** devolve uma cópia imutável; **DebugPanel** desenha a ~60 Hz.

Por que Swing? Já acompanha o JDK 17 e permite visualizar e calibrar a jogabilidade sem adicionar JavaFX, LibGDX ou uma engine 2D. É mais rápido e com menos dependências para este estágio; a apresentação será substituída mais tarde.

Por que não integrar Kryonet agora? O cliente visual já valida as regras sem acoplar domínio e UI à rede. O próximo módulo poderá transformar pacotes de entrada em DebugInput; exigiríamos testes adicionais de concorrência e sincronização antes de declarar a versão em rede pronta.

4. Regras implementadas

Movimento

Velocidade base de 215 px/s, normalização de vetores diagonais, integração com delta fixo de 1/30 segundo e deslocamento por eixos para deslizar nas paredes.

Colisão física

Círculos para Player (raio 18 px) e Mob (raio 19 px); paredes retangulares; limites do mundo. A função de elegibilidade barra deslocamentos que gerariam interseção.

Ataque básico

Direção do último movimento (8 direções), cone de 90°, alcance de 74 px + raio do alvo, cooldown de 0,38 segundo; golpe sem consumo de vigor.

Dano e HP

PlayerArchetype humano sem arma/estilo utiliza UnarmedStrike (força/2). CombatService mantém a afinidade racial, aplica dano e publica EntityDeathEvent. HP é calculado por VitalityService.

IA dos inimigos

Busca reativa do jogador a até 320 px; movimentação a 92 px/s; ataque próximo a 58 px, com cooldown de 1,15 segundo. Não existe pathfinding A*.

5. Inventário completo de mudanças

Caminho	Mudança
server/pom.xml	JUnit 5 (test), plugins Surefire e Exec, Java release 17/UTF-8.
DatabaseManager.java	Credenciais JDBC removidas do fonte; usa variáveis SPIRIT_DB_*.
.gitignore	Ignora target, saída de compilação e arquivos de segredos locais.
README-PROTOTIPO.md	Guia dos controles, comandos de execução e limitações.
CHANGELOG-PROTOTIPO.md	Registro técnico de cada adição ou alteração.
rodar-prototipo.cmd	Comando Windows de 1 clique: testa e abre a janela.
prototype/DebugInput.java	Input imutável com direções e ataque.
prototype/DebugSnapshot.java	DTOs imutáveis para atores, obstáculos e danos.
prototype/DebugWorld.java	Mundo, fábricas, colisões, movimento, inimigos e combate.
prototype/DebugRuntime.java	Game loop isolado e publicação segura de snapshots.
prototype/DebugPanel.java	Desenho da arena, atores, barras, efeitos e hitboxes.
prototype/DebugLauncher.java	Janela Swing e Key Bindings.
tests/DebugWorldTest.java	5 cenários JUnit 5 que rodarão com Maven.
tests/DebugWorldSelfTest.java	6 validações standalone executadas sem Maven.

Preservado sem alteração: todos os fontes em com.projetok.domain.*, KServer, Kclient, Main e Network. Nenhum arquivo original foi apagado ou renomeado.

6. Validação e resultados de testes

Verificação	Estado	Evidência
OpenJDK 21 com javac --release 17	PASSOU	Domínio, demo e protótipo compilados como código Java 17.
Smoke test: mundo e HP	PASSOU	1 Player + 3 Mobs; HP 100/120.
Smoke test: diagonal	PASSOU	Velocidade diagonal equivalente à horizontal.
Smoke test: colisão	PASSOU	Parede, bordas e corpos sólidos.
Smoke test: dano e cooldown	PASSOU	Golpe de 15 HP e proteção contra repetição antes do cooldown.
Smoke test: contra-ataque	PASSOU	Mob alcança e danifica o jogador.
Smoke test: morte e snapshots	PASSOU	Evento único ao derrotar mob e lista imutável.
Janela Swing sob Xvfb	PASSOU	Janela aberta; teclas simuladas; captura em PNG.
mvn test / JUnit 5	NÃO EXECUTADO	Maven e JARs externos indisponíveis neste ambiente.
Rede Kryonet / Postgres	FORA DO ESCOPO	Não usados na arena local; o legado permanece separado.

Transparência: a janela gráfica foi de fato executada no ambiente de geração. O build completo via Maven, a suíte JUnit 5 e o script .cmd serão validados quando você rodar no Windows. Não afirmamos que a integração Kryonet está pronta.

7. Como executar no seu computador

1. Extraia o arquivo **Spirit_Gate_Prototipo_Jogavel.zip** e abra a pasta **Projeto-K**.
2. Confira o Java 17 e o Maven já configurados no seu Windows.
3. Dê duplo clique no arquivo **rodar-prototipo.cmd** (o script roda os testes primeiro).
4. Aguarde o download de dependências de teste na primeira execução e use WASD/Espaço/H/R.

Alternativa pelo CMD (na pasta server):

```
mvn clean test
mvn exec:java -Dexec.mainClass=com.projetok.prototype.DebugLauncher
```

8. Segurança, limites e próximos passos

Alerta sobre senha anteriormente exposta

O arquivo DatabaseManager tinha uma senha explícita no pacote original. Ela foi removida do código entregue, mas pode permanecer no histórico do seu Git. **Troque a senha no PostgreSQL**, nunca a reutilize e avalie a limpeza do histórico com cautela. A arena funciona sem banco de dados.

Backlog que NÃO faz parte desta entrega

- Login real, sessões Kryonet e múltiplos jogadores sincronizados.
- IA com navegação ao redor de obstáculos, mapas maiores e índice espacial.
- Stamina física, stamina XP, habilidades, congelamento e zona persistente.
- Persistência de posição, equipamentos, economia, ranking e barcos.
- Tratamento de latência, anti-cheat avançado, interpolação e cliente final.

Próxima iteração recomendada

1. Execute os testes Maven no Windows. **2.** Caminhe pela arena e relate qualquer colisão ou ataque inesperado. **3.** Conecte o loop isolado a comandos da rede Kryonet sem permitir que a thread de rede modifique GameEntity. **4.** Decida direção/alcance definitivo antes de expandir combate e skills.

Mentoria retomada: como primeira tarefa sua, altere apenas o valor WALK_SPEED no DebugWorld, execute novamente e compare velocidade e colisões. Depois podemos construir juntos um quarto mob ou obstáculo, com testes e revisão.

Documento acompanhado de: ZIP do código completo atualizado, screenshot PNG da arena e README-PROTOTIPO.md dentro do repositório.